

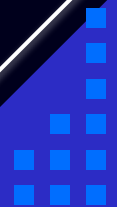
Proyecto Innovatech: Arquitectura de Microservicios e Integración Frontend

Asignatura: Desarrollo Fullstack III (DSY1106)

Integrantes: John Beiza, Dante Torres

■ **El Desafío:** Construir un sistema escalable y seguro capaz de gestionar usuarios, controlar el acceso y procesar métricas de monitoreo en tiempo real.

La Solución Propuesta: Se implementó una arquitectura distribuida que divide las funcionalidades del sistema en un API Gateway (BFF) y microservicios independientes.

- **ms-recursos:** Centraliza la seguridad (JWT) y la persistencia de usuarios.
 - **ms-monitoreo:** Maneja la analítica y consume eventos asíncronos en tiempo real mediante mensajería (RabbitMQ).
 - **ms-proyectos:** Se encarga de la gestión de proyectos, incorporando modelos DTO y controladores REST estándar.
 - **Api-gateway:** Maneja todo el enrutamiento de microservicios y permite una comunicación segura con el frontend. Además, implementa un circuit breaker que promueve disponibilidad.
- 

Patrón Utilizado: Factory Method en la capa de servicios.

■ **¿Qué es?** Un patrón creacional que define una interfaz común para la creación de objetos, pero delega en subclasses específicas la decisión de qué clase concreta instanciar.

Aplicación en el Sistema: Se implementó en el motor de monitoreo para desacoplar la lógica al momento de generar dinámicamente distintos tipos de reportes, alertas o métricas. El cliente solo solicita la métrica a la "fábrica" y trabaja contra una interfaz común.

Beneficios:

- Reduce el acoplamiento del código.
- Facilita la escalabilidad: permite agregar nuevos canales o tipos de métricas a futuro sin modificar el código cliente existente.

IMPLEMENTACIÓN: circuit breaker

Se incorporó un circuit breaker en el api gateway con tal de permitir al sistema sostenerse en pie en el evento de que uno de los servicios no se encontrase en línea. El circuit breaker permite manejar varios estados de aplicación, siguiendo la lógica de un circuito cerrado, abierto, o semi-abierto.

Contribuye a un sistema mantenible y de alta disponibilidad.

IMPLEMENTACIÓN: RABBITMQ

Se utilizó RabbitMQ como Message Broker para satisfacer los requerimientos que se refieren a eventos a tiempo real. RabbitMQ permite configurar a un servicio como consumidor de eventos y a otro como productor de estos, así como permite establecer qué eventos serán monitoreados.

Esta herramienta fue clave para complacer los requerimientos relacionados al servicio de monitoreo.

Frameworks Principales

- Spring Boot: Se utilizó el framework Spring, así como varias otras herramientas de su ecosistema (JPA, WebFlux, FeignClient, Cloud, Security, entre otros) para la construcción del Backend y el BFF.
- React: Se utilizó en conjunto con algunas de sus librerías (react hook forms; etc) para el front end.

Justificación:

- Spring: Spring es el framework por defecto para el desarrollo con java de microservicios, debido a ofrecer el paquete completo en lo que se refiere a dependencias y compatibilidad.
- React: Permite crear aplicaciones óptimas mediante el uso de componentes y pages, consumiendo menor memoria y recursos operativos.

componentes ci/cd

En lo que se refiere al área de CI/CD, se utilizaron las siguientes herramientas:

- Swagger (relacionado): para la documentación accesible de APIs.
- Sonarqube: Para el testeo de la calidad del código. Esta herramienta permite visualizar cómo rinde nuestro código en esferas determinadas, lo cual es importante de considerar cuando se busca optimizar.
- Docker: Para el despliegue continuo, se emplean contenedores docker que permiten un entorno de desarrollo y operación constante y confiable.
- Eureka server: Se encarga de centralizar el manejo de los microservicios.

UNIT TESTING

Se implementó unit testing, buscando un 60% de cobertura respecto a las capas de servicio presentes en el backend. Para este fin, se utilizó JUnit y Mockito, y se testeó cada método definido en cada servicio implementado.

GIT FLOW

Se trabajó con Git Flow como flujo de trabajo, siguiendo la estructura de separar features de hotfixes de releases mediante ramas. El flujo de trabajo fue en su mayor parte desacoplado, y se hizo uso de funciones básicas e integrales de git como lo es el branching, los worktrees, el merging, entre otros.

conclusión

Para esta tercera entrega se debió incorporar otros patrones de arquitectura, así como distintos componentes con enfoque a la seguridad y operatividad del sistema. También se debió dar importancia a las fases de integración y despliegue de una aplicación, lo cual se refleja al haber trabajado con sonarqube, docker y unit testing. Aún así, algo que no se incorporó fueron elementos como pipelines o cualquier uso de github actions para un flujo de trabajo aún más automatizado.